

SCS-RG: Smart Campus Safety & Risk Grid

Backend Architecture, Setup Guide, API Reference, and Risk Fusion Specification

Team Clover

Sadman Islam • Shah Samin Yasar • Ahmed Thousif Thisham

Metropolitan University

Web Dashboard: <https://clover-scs-rg.vercel.app/>

Backend API Base: <https://robofusion-techathon-teamclover.onrender.com>

Abstract: This document serves as the comprehensive technical documentation for the backend service of the **Smart Campus Safety & Risk Grid (SCS-RG)**. Built for real-time IoT multi-hazard environmental monitoring, automated incident dispatching, and predictive risk management across university campus infrastructure, the SCS-RG backend features a bounded multi-hazard risk fusion engine, real-time WebSocket state distribution, atomic single-write concurrency controls, a logistic regression predictive model, and a natural-language incident parsing engine. This document details the software architecture, installation procedures, mathematical formulas, Prisma database schema, and complete API endpoint specifications with example HTTP payloads.

1. Software Architecture Overview

The SCS-RG backend is engineered using a modular, decoupled architecture built on Node.js, Express.js, TypeScript, Prisma ORM, PostgreSQL, and Socket.io.

1.1 Architectural Layers

The application enforces strict separation of concerns through a layered Controller-Service-Repository pattern:

- **Transport Layer (HTTP & WebSockets):** Handles REST endpoints via Express controllers and real-time event streaming via Socket.io.
- **Middleware Layer:** Enforces role-based access control (RBAC), session authentication (`requireSession`, `requireAdmin`), and microcontroller hardware authentication (`validateZoneKey`).
- **Service Layer (Domain Logic):** Contains core algorithms including the Bounded Multi-Hazard Risk Fusion Engine, Flame Debounce Filter, Exponential Smoothing, Priority Ranking Engine, ML Predictor, and Natural Language Report Parser.
- **Persistence Layer (ORM & Database):** Interfaces with PostgreSQL using Prisma ORM for structured queries and transactional safety.

1.2 Realtime Event Distribution (Socket.io)

The backend maintains low-latency event broadcasting to all connected command center dashboards:

- `zone:state` : Broadcasts real-time risk score, hazard contributions (fire, gas, water, occupancy), and zone state updates.
- `priority:update` : Broadcasts the re-ranked priority dispatch queue whenever an incident opens, changes severity, or is acknowledged.
- `incident:opened`, `incident:acked`, `incident:resolved` : Broadcasts lifecycle incident events to trigger real-time audio/visual toast notifications on operator consoles.
- `zone:offline` : Broadcasts heartbeat expiration events when a hardware node stops reporting for >10 seconds.

2. Setup & Execution Guide

2.1 Environment Variables Configuration

Create a `.env` file inside the `backend/` root directory with the following variables:

```
PORT=4000
NODE_ENV=production
DATABASE_URL="postgresql://user:password@localhost:5432/scs_rg_db"
FRONTEND_URL="https://clover-scs-rg.vercel.app"
BETTER_AUTH_SECRET="your-256-bit-secret-key"
BETTER_AUTH_URL="https://robofusion-techathon-teamclover.onrender.com"
```

2.2 Step-by-Step Installation Commands

```
# 1. Navigate to backend directory
cd backend

# 2. Install dependencies
npm install

# 3. Apply Prisma schema to PostgreSQL database
npx prisma db push

# 4. Seed initial zones and admin/staff user accounts
npx prisma db seed

# 5. Build TypeScript source code
npm run build

# 6. Start production server
npm start
```

2.3 Demo Reset Procedure

To restore seeded zone baselines and clear synthetic test readings generated during testing, run:

```
npm run reset:demo
```

3. Mathematical Risk Calculation Formula & Fusion Engine

The SCS-RG backend evaluates zone risk using a **Bounded Multi-Hazard Risk Fusion Engine**. It calculates a single scalar Risk Score $R \in [0.0, 100.0]$ derived from raw sensor telemetry.

3.1 Sensor Telemetry Inputs

Sensor Type	Raw Input Range	Description
Flame Sensor	0 - 4095 (12-bit) / 0 - 1023 (10-bit)	Infrared flame detector signal
Gas Sensor (MQ2)	0 - 4095 (12-bit) / 0 - 1023 (10-bit)	Combustible gas / smoke concentration
Water Level	0 - 4095 (12-bit) / 0 - 1023 (10-bit)	Submersion / flood level sensor
PIR Motion	0 or 1	Passive infrared human occupancy sensor

3.2 Dynamic ADC Resolution Scaling

To maintain compatibility across hardware platforms (12-bit ESP32 vs 10-bit Arduino), maximum ADC resolution is dynamically detected per cycle:

$$\begin{aligned} \text{MaxADC}(x) &= 4095.0 \text{ if } x > 1023 \text{ else } 1023.0 \\ x_{\text{gas}} &= \min(1.0, \text{gas_raw} / \text{MaxADC}(\text{gas_raw})), \quad x_{\text{water}} = \min(1.0, \text{water_raw} / \\ &\quad \text{MaxADC}(\text{water_raw})) \end{aligned}$$

3.3 Flame Debounce Filter (N = 3 Cycles)

To prevent false triggers from optical noise, raw flame signals must pass a 3-cycle consecutive window before activating full weight:

$$x_{\text{fire_norm}} = 1.0 \text{ (if debounced signal is ACTIVE) else } \min(1.0, \text{flame_raw} / \text{MaxADC}(\text{flame_raw}))$$

3.4 Risk Fusion Weights & Hard Bound

Hazard Component	Max Weight (w_i)	Contribution Calculation
Fire (C_{fire})	40.0	$40.0 \times x_{\text{fire_norm}}$
Gas (C_{gas})	40.0	$40.0 \times x_{\text{gas_norm}}$ (zeroed during 30s warm-up)
Water (C_{water})	30.0	$30.0 \times x_{\text{water_norm}}$
Occupancy (C_{occ})	25.0	$25.0 \times \text{motion}$

The total Risk Score R is calculated and bounded at 100.0:

$$R = \min(100.0, C_{\text{fire}} + C_{\text{gas}} + C_{\text{water}} + C_{\text{occ}})$$

3.5 Zone State Classification Thresholds

- **SAFE:** Risk Score < 30.0
- **WARNING:** $30.0 \leq \text{Risk Score} < 65.0$
- **CRITICAL:** Risk Score ≥ 65.0
- **OFFLINE:** Sensor health disconnected OR heartbeat silent > 10s

4. Database Schema Design (Prisma ORM)

```
model Zone {
  id          String      @id
  name        String
  hazardProfile String
  apiKey       String      @unique
  state        String      @default("SAFE")
  riskScore    Float       @default(0.0)
  lastSeenAt   DateTime?
  archived     Boolean      @default(false)
  readings     Reading[]
  incidents     Incident[]
}

model Reading {
  id          String      @id @default(cuid())
  zoneId      String
  seq         Int
  riskScore    Float
  flameRaw     Int
  gasRaw       Int
  waterRaw     Int
  motion       Boolean
  receivedAt   DateTime @default(now())
  zone        Zone       @relation(fields: [zoneId], references: [id])
}

model Incident {
  id          String      @id @default(cuid())
  zoneId      String
  status      String      @default("OPEN")
  hazardTypes String[]
  peakRiskScore Float
  source      String      @default("sensor")
  openedAt    DateTime    @default(now())
  acknowledgedBy String?
  acknowledgedAt DateTime?
  resolvedAt  DateTime?
  zone        Zone       @relation(fields: [zoneId], references: [id])
  transitions IncidentTransition[]
}

model IncidentTransition {
```

```
id          String    @id @default(cuid())
incidentId  String
fromState   String?
toState     String
riskScore   Float
occurredAt  DateTime @default(now())
incident    Incident @relation(fields: [incidentId], references: [id])
}
```

5. API Endpoints Reference with Example Requests & Responses

5.1 Telemetry Ingestion (POST /api/readings)

Ingests sensor readings from microcontroller nodes. Authenticated via header `X-Zone-Key`.

```
POST /api/readings HTTP/1.1
Host: robofusion-techathon-teamclover.onrender.com
Content-Type: application/json
X-Zone-Key: key_server_room_456

{
  "zone_id": "server_room",
  "seq": 142,
  "timestamp_ms": 1721985430000,
  "sensors": {
    "flame_raw": 3200,
    "gas_raw": 1800,
    "water_raw": 150,
    "motion": true
  },
  "sensor_health": {
    "flame": "ok",
    "gas": "ok",
    "water": "ok",
    "motion": "ok"
  }
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Reading ingested successfully",
  "data": {
    "accepted": true,
    "state": "CRITICAL",
    "risk_score": 73.2,
    "commands": {
      "led": "red",
      "buzzer": true,
      "relay_cutoff": true
    },
    "server_seq_ack": 142
  }
}
```

5.2 Get All Active Zones (GET /api/zones)

Response (200 OK):

```
{
  "success": true,
  "message": "Zones retrieved successfully",
  "data": [
    {
      "zone_id": "server_room",
      "name": "Server Room 101",
      "hazard_profile": "High density electrical & server racks",
      "state": "CRITICAL",
      "risk_score": 73.2,
      "occupied": true,
      "last_seen_at": "2026-07-26T14:30:00.000Z"
    }
  ]
}
```

5.3 Priority Dispatch Queue (GET /api/priority)

Response (200 OK):

```
{
  "success": true,
  "message": "Priority queue retrieved successfully",
  "data": {
    "ranked": [
      {
        "zone_id": "server_room",
        "rank": 1,
        "risk_score": 73.2,
        "occupied": true,
        "seconds_critical": 45,
        "reason": "risk 73.2, occupied, critical 45s",
        "source": "sensor",
        "incident_id": "clx82910a0001"
      }
    ]
  }
}
```

5.4 Acknowledge Incident (POST /api/incidents/:id/ack)

Response (200 OK):

```
{
  "success": true,
  "message": "Incident acknowledged successfully",
  "data": {
    "success": true,
    "statusCode": 200,
    "incident_id": "clx82910a0001",
    "acknowledged_by": "usr_admin_123",
  }
}
```

```
    "acknowledged_at": "2026-07-26T14:31:00.000Z"
  }
}
```

5.5 Logistic Regression Risk Predictor (GET /api/bonus/ml-predict/:zone_id)

Response (200 OK):

```
{
  "success": true,
  "message": "ML risk prediction calculated successfully",
  "data": {
    "zone_id": "server_room",
    "predicted_critical_probability": 0.885,
    "predicted_critical_next_5m": true,
    "model": "logistic_regression_v1",
    "safety_guarantee": "Predictor is strictly advisory and cannot trigger
relay/buzzer actuation"
  }
}
```

5.6 Natural Language Report Ingestion (POST /api/bonus/nl-report)

Request:

```
{
  "text": "Heavy smoke and electrical fire smell coming from the server room rack 4"
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Natural language report processed successfully",
  "data": {
    "parsed": true,
    "input_text": "Heavy smoke and electrical fire smell coming from the server room
rack 4",
    "extracted_signal": {
      "zone_id": "server_room",
      "hazard_type": "fire",
      "estimated_severity": "CRITICAL"
    },
    "validation_gate": "passed",
    "incident_id": "clx99201b0002"
  }
}
```

6. System Quality Assurance

Test ID	Verification Metric	Status
F1	10s Inactivity Sweep → Zone transition to OFFLINE	PASS
F2	N=3 Debounce & Exponential Decay Smoothing	PASS
F3	X-Zone-Key Header Validation against Payload zone_id	PASS
TC6b	Out-of-range sensor payload rejection (HTTP 400)	PASS
TC7b	Atomic Concurrent Ack Race Condition (1x 200, 9x 409)	PASS
TC13b	Server-side RBAC Enforcement (requireAdmin)	PASS